JAŃA
POST

JANA POST
× TERRICON VALLEY
HACKATHON

JANA POST

Международная технологическая и логистическая платформа, обеспечивающая полный цикл трансграничной торговли между Китаем и странами Центральной Азии.

Компания объединяет собственную логистическую инфраструктуру, цифровые решения и экосистему сервисов для клиентов, предпринимателей, маркетплейсов и бизнеса.

КИТАЙ

↓

КАЗАХСТАН

↓

ЦЕНТРАЛЬНАЯ АЗИЯ

Кыргызстан · Узбекистан
Таджикистан · Монголия

3+ млн
пользователей

1000+
филиалов

5
стран

IT
собственная платформа

В основе Jana Post лежат три ценности:

01 ОТКРЫТОСТЬ

02 НАДЁЖНОСТЬ

03 РЕЗУЛЬТАТ

01

JANA ADDRESS

Разобрать произвольный адрес на точные поля для логистики.

02

DIGITAL HR

Собрать профиль, заявки, обучение и HR-помощь в одном пространстве.

АДРЕСЫ ПИШУТ КАК УГОДНО

«66квартал 12дом 69квартира»

«Туран 55/6 магазин береке 4подьезд 9этаж»

НА ВЫХОДЕ

Область · город · район · улица · дом
Квартира · подъезд · этаж · ориентир

Точность разбора + восстановление недостающих полей по собственному справочнику.

РАБОТАЮЩИЙ ПАЙПЛАЙН

01 Разбор	Все записи, отдельные поля и confidence
02 Справочник	Собственный источник районов и жилых массивов
03 Проверка	REST API для пакета + экран ручной правки
04 Запуск	docker compose up · автономно без сети

45% метрики на скрытых адресах

30% инженерия 15% интерфейс 10% презентация

Задача

Клиент вводит адрес доставки одной строкой, как ему удобно: «66квартал 12дом 69квартира»,

«Ул .Курильская 9 кв 1», «Туран 55/6 магазин береке 4подьезд 9этаж».

Нужно разобрать такую строку на отдельные поля: область, город, район, улица, дом, квартира, подъезд, этаж.

Сейчас адрес хранится в базе как есть, одной строкой. Из-за этого не работает аналитика по городам, не считается автоматически зона доставки, а курьер уточняет квартиру и подъезд по телефону.

Район города и жилой массив в части строк указаны прямо — их нужно распознать наравне с улицей. В остальных строках их нет, и восстановить их можно только по справочнику. **Справочник организаторы не выдают — команда находит источник и разворачивает его сама.** Это часть задачи: оценивается решение целиком, а не только разбор текста.

Возможные источники: OpenStreetMap (дамп по Казахстану), data.egov.kz, справочник КАТО, открытые геокодеры. Выбор и способ подготовки — на усмотрение команды.

Данные

Выдаются на старте:

Файл	Что внутри
train.json	Несколько тысяч реальных адресных строк из базы Jana Post. Формат тот же, что у sample_input.json
sample_input.json	Пример входного файла — в таком виде жюри подаст тестовый набор
sample_output.json	Пример правильного ответа

Формат

Вход — одна строка на запись. Область, город и сам адрес склеены вместе, как они лежат в базе:

```
{
  "id": "rec_00417",
  "raw_address": "город Алматы/Алматы қаласы, Мкр Аксай 3а, 59, кв 70"
}
```

Строка может содержать двуязычные склейки через слэш, повтор города, а вместо города — район. Разбирать это тоже задача решения.

Ответ:

```
{
  "id": "rec_00417",
  "raw_address": "город Алматы/Алматы қаласы, Мкр Аксай 3а, 59, кв 70",
  "province": "Алматы",          // область или город респ. значения
  "city": "Алматы",
  "district": "Ауэзовский",      // из справочника: в строке района нет
  "living_area": null,           // микрорайон или ЖК, если улица указана отдельно
  "street_type": "микрорайон",   // улица / проспект / микрорайон / переулок...
  "street_name": "Аксай 3а",     // без типа улицы, ровно как в строке
  "house": "59",
  "apartment": "70",
  "entrance": null,
  "floor": null,
  "landmark": null,              // ориентир, если был в строке
  "confidence": 0.91             // насколько уверены, 0..1
}
```

raw_address возвращается без изменений — так проще сверять разбор с исходником.

Тип улицы возвращается отдельным полем от названия. Это нужно, чтобы сравнение с эталоном было однозначным: «улица Абая» и «проспект Абая» — разные улицы, а «пр. Абая» и «проспект Абая» — одна и та же.

Пропускать записи нельзя — на каждую входную запись должен быть ответ. Если разобрать не удалось, возвращается объект с null в полях и низким confidence.

Правила заполнения полей

Поле	Как заполнять
province	Название в прямом порядке: «Актюбинская область», «Алматы» для городов респ. значения
city	Только название: «Актобе», «Боралдай». Без слов «город», «пос.», без казахской половины
district	Название без слова «район»: «Илийский», «Ауэзовский». Если не определён — null
living_area	Микрорайон или ЖК, когда улица указана отдельно: «Пришахтинск», «Томирис»
street_type	Тип отдельным словом: улица, проспект, микрорайон, переулок, квартал. Если в строке не указан и определить нельзя — null
street_name	Название без типа, ровно как в строке. Падеж не меняем, опечатки не исправляем: «казибек би» остаётся «казибек би»
house	Номер дома с дробью и литерой: «1/2», «12а». «1/2» и «1» — разные значения
apartment / entrance / floor	Только номер, без слов «кв», «подъезд», «этаж»
landmark	Ориентир, если он есть в строке: «магазин Береке»
confidence	Оценка уверенности 0..1. В метрику не входит, смотрится на защите

Поле, которое не удалось определить, возвращается как null. Записи, где поля нет ни в строке, ни в эталоне, из подсчёта по этому полю исключаются — они не засчитываются ни как верные, ни как ошибочные.

Что ещё нужно сдать

- REST API с методом пакетной обработки — через него жюри прогонит тестовый набор
- Простой интерфейс: загрузить файл, посмотреть разбор, вручную поправить неуверенные записи
- Запуск одной командой: docker compose up
- README с инструкцией

На что обратить внимание

Реальные сложности из данных:

- Слитное написание: «66квартал», «Назарбаева8», «11дом», «4подьезд»
- Дробные и литерные номера домов: 1/2, 33/1, 55/6, 12а — дробь терять нельзя
- Латиница вперемешку: «flat 191», «kv 12»
- Номер микрорайона легко спутать с номером дома: «Нурсити 2 микрорайон 11дом 5 подьезд 239 кв»
- Ориентиры прямо в адресе: «магазин береке», «Qazpost Солнечный»
- Казахский и русский вперемешку через слэш: «город Актобе/Ақтөбе қаласы»
- Город может дублироваться в строке, а вместо города может стоять район
- Опечатки и лишние пробелы: «Ул .Курильская», «подьезд» вместо «подъезд»
- Невидимые служебные символы внутри строк

Как оцениваем

Жюри прогоняет через каждое решение скрытый набор адресов, размеченных вручную. Метрики:

Метрика	Что означает	Вес
Точность адреса	Верно определены область, город, название улицы и дом	60%
Точность деталей	Верно извлечены квартира, подъезд, этаж	25%
Точность справочных полей	Верно извлечены район и жилой массив — только по записям, где они есть в строке	15%

street_type в метрику не входит. В части строк тип улицы не написан вовсе («Кулманова 133», «жуалы 19»), и определить, улица это или проспект, можно только по справочнику. Поэтому оценивается название улицы, а тип смотрит жюри на защите.

Район и жилой массив в одних строках указаны, в других нет: «Турксибский район, Рыбинский переулок, дом 4» — есть, «жуалы 19» — нет. Эти два случая считаются по-разному.

Ситуация	Как оценивается
Данные есть в строке, решение их извлекло	Засчитывается в метрику
Данные есть в строке, решение вернуло null или неверное значение	Ошибка разбора, снижает метрику
Данных в строке нет, решение вернуло null	Не штрафуется, в метрику не входит
Данных в строке нет, решение подняло их из справочника	В метрику не входит, засчитывается жюри на защите

То есть автоматически проверяется только разбор того, что написано в строке. Восстановление недостающих полей по своему справочнику жюри смотрит на защите: насколько полно они заполнены, откуда взяты данные и как устроено сопоставление. Это основная часть инженерного критерия.

Сравнение по каждому полю отдельно и посимвольно: приводим к нижнему регистру, убираем пунктуацию и лишние пробелы, сравниваем. Падежи и опечатки не прощаются — название возвращается в том виде, в каком оно написано в исходной строке. Номера домов и квартир сравниваются как строки: «1/2» и «1» это разные значения.

Итоговая оценка команды:

Критерий	Вес
Метрики на скрытом наборе	45%
Инженерная часть: справочные поля, сбор справочника, архитектура, качество кода	30%
Удобство интерфейса	15%
Презентация	10%

Правила

- Стек любой, упаковка в Docker обязательна
- Во время работы над решением интернет доступен без ограничений: можно качать дампы, пользоваться геокодерами и языковыми моделями для подготовки своей базы
- Во время финального прогона сеть отключена. Решение должно работать автономно — все справочники и модели лежат внутри образа
- Разбор должен укладываться в 5 минут на 1000 записей. Загрузка справочников при старте в лимит не входит, но запуск контейнера не должен занимать больше 3 минут
- Выданные данные не выносятся за пределы хакатона и удаляются после защиты
- Словари, справочники и таблицы соответствий разрешены и приветствуются — это и есть решение задачи. Запрещено другое: сопоставление по id записи и возврат готового ответа по точному совпадению всей входной строки. Дисквалификация

ОДНА ПЛАТФОРМА ДЛЯ СОТРУДНИКА

Сейчас профиль, заявления, обучение и ответы HR находятся в разных местах. Команда показывает единый сквозной путь.

01	02	03	04	05
ВХОД И РОЛЬ	ПРОФИЛЬ	ЗАЯВКА	ОБУЧЕНИЕ	HR-АССИСТЕНТ

Заявка «Отпуск» или «Аванс» → статус и уведомление. Курс → прогресс. Ответ ассистента → ссылка на источник.

ПРОТОТИП ЗА 56 ЧАСОВ

3 РОЛИ

Сотрудник · руководитель
HR / администратор
Доступ по правам.

1 ЗАЯВКА

Отпуск или аванс
Статус, история, уведомление.

AI + LMS

Курс с прогрессом
HR-ответ по базе знаний
со ссылкой на источник.

Без реальных персональных данных и выдуманных AI-ответов.

Mock API допустимы. Покажите место будущих интеграций с 1С:ЗУП и Documentolog.

30% сценарий 20% UX 20% архитектура 15% AI 10% Academy 5% защита

Задача

Внутренние HR-сервисы компании распределены между несколькими системами и каналами. Работнику приходится отдельно искать кадровые данные, оформлять заявления, отслеживать статусы, проходить обучение и обращаться в HR за типовыми разъяснениями. Для HR и руководителей это означает дополнительную ручную координацию, повторяющиеся вопросы и отсутствие единой точки взаимодействия с работником. Нужно создать прототип единой цифровой HR-платформы: работник входит в одно пространство, видит свой профиль и ключевые HR-сервисы, может оформить типовую заявку, пройти обучение и получить помощь у AI ассистента. Решение должно показать сквозной пользовательский путь и ролевую модель, а не пытаться за 56 часов воспроизвести полноценную промышленную HR систему. Что должна сделать команда

Что должна сделать команда

Контур	Минимальный результат
Единый вход и роли	Авторизация в платформе и минимум 3 роли: работник, руководитель, HR/администратор. Пользователь видит только разрешенные ему разделы и данные.
Личный кабинет	Профиль работника: Ф.И.О., компания, должность, подразделение, руководитель, дата приема, остаток отпуска, назначенное обучение и уведомления.
HR-сервис	Минимум один рабочий сценарий заявки: «Отпуск» или «Аванс». Пользователь создает заявку и видит ее статус и историю.
Мои заявки	Единый список заявок с типом, датой, текущим статусом и карточкой заявки.
Jana Academy	Минимальный LMS-сценарий: каталог/назначенный курс, просмотр материала, фиксация прогресса и результата.
AI Jana	HR-ассистент (назовем AIJana) отвечает на типовые вопросы только по предоставленной/подготовленной базе знаний, показывает источник ответа и корректно сообщает, если надежного ответа нет.
Уведомления	Центр уведомлений минимум для изменений статуса заявки и назначения/завершения обучения.

Границы хакатонного MVP

Промышленные интеграции с 1С:ЗУП и Documentolog в рамках хакатона не требуются. Команда может использовать mock API / тестовые данные, но должна показать, где и как в архитектуре подключаются внешние системы. Официальные кадровые данные считаются read-only: пользователь не должен менять их напрямую.

- Не требуется полноценное мобильное приложение — достаточно адаптивной веб-версии.
- Не требуется электронная подпись и хранение оригиналов кадровых документов.
- Не требуется реализовывать рекрутинг, Performance Management и все будущие HR-модули.
- Не требуется принимать кадровые решения с помощью AI.
- Персональные данные других работников не должны быть доступны пользователю без соответствующей роли.

Тестовые данные

Для демонстрации команда формирует тестовый набор пользователей и данных, достаточный для проверки ролей и сквозных сценариев. Минимально рекомендуются: работник, руководитель и HR/администратор; несколько заявок в разных статусах; 1–2 учебных курса; небольшая база знаний для AI Jana. Реальные персональные данные работников использовать не требуется.

Ключевые сценарии демонстрации

№	Сценарий
1	Работник входит в JanaMen и видит свой профиль, быстрые сервисы, заявки, обучение и уведомления.
2	Работник создает заявку на отпуск или аванс и видит изменение ее статуса.
3	Руководитель видит разрешенную информацию по своей команде/заявке, но не получает доступ к чужим данным.
4	Работник открывает назначенный курс, проходит материал/тест и видит прогресс.
5	Работник задает AI Jana типовой HR-вопрос и получает ответ со ссылкой на источник.
6	На вопрос вне базы знаний AI Jana не выдумывает ответ, а сообщает об отсутствии надежной информации/предлагает эскалацию в HR.

Что нужно сдать

- Работающий веб-прототип / MVP.
- Исходный код в репозитории.
- README с инструкцией по запуску, тестовыми учетными записями и описанием реализованных сценариев.
- Схему архитектуры с указанием места будущих интеграций с 1С:ЗУП и Documentolog.
- Короткую презентацию решения и демонстрацию сквозного пользовательского сценария.

На что обратить внимание

- **Простота:** основные действия работника должны быть понятны без отдельной инструкции и выполняться за минимальное число шагов.
- Ролевая модель: ограничения доступа должны работать не только визуально в интерфейсе.
- Единый UX: профиль, заявки, обучение, уведомления и AI должны ощущаться как одна платформа, а не набор несвязанных экранов.
- AI-безопасность: AI Jana не должен придумывать кадровые правила или раскрывать чужие данные.
- Расширяемость: архитектура должна позволять позже подключить реальные 1С:ЗУП, Documentolog и дополнительные HR-модули.
- RU/KZ: приветствуется возможность переключения языка интерфейса или архитектурная готовность к локализации.

Как оцениваем

Критерий	Что оценивается	Вес
Сквозной пользовательский сценарий	Работают ключевые пути: вход → профиль → заявка → статус/уведомление; обучение и AI интегрированы в общий опыт.	30%
Функциональность и UX	Понятность интерфейса, логика кабинета, минимум лишних действий, адаптивность.	20%
Архитектура и инженерная часть	Роли и права, структура данных, API, качество кода, готовность к будущим интеграциям.	20%
AI Jana	Ответы по базе знаний, указание источника, корректное поведение при отсутствии ответа, соблюдение ролевого доступа.	15%
Jana Academy	Курс, прохождение, прогресс/результат и связь с профилем работника.	10%
Презентация	Ясность решения, демонстрация ценности для работника и компании.	5%

Правила

- Стек технологий — на усмотрение команды.
- Допускаются mock API и тестовые данные для 1С:ЗУП и Documentolog; способ будущей интеграции должен быть описан.
- Решение должно запускаться по понятной инструкции; Docker / docker compose приветствуется.
- Использование AI-моделей и внешних библиотек разрешено.
- В репозитории не должно быть реальных персональных данных, паролей, токенов и секретов.
- Команда должна явно указать, какие части реализованы полностью, какие смоделированы, а какие оставлены как дальнейшее развитие.

Контекст будущего продукта

Полная концепция JanaMen предполагает единую корпоративную HR-платформу с интеграцией с 1С:ЗУП и Documentolog, личным кабинетом, HR-заявками, уведомлениями, Jana Academy и безопасным AI Jana. Хакатонный кейс намеренно ограничен прототипом, чтобы команда могла за время мероприятия показать качество продуктового и технического решения.

JAŃA
POST

УДАЧИ,
КОМАНДЫ!

JANA POST × TERRICON VALLEY